Home Sleep Apnea Screening

Team: Barbara Klimek Tomatova, Aliénor Franck de Préaumont

GitHub: https://github.com/BarbaraKlimekTomatova/ACC1ILV-cloud-and-AI-infrastructure-for-sleep-apnea-detection-

What the project is about

Situation: People who might have sleep apnea often wait months for a night in a sleep lab. Instead, they could take a small device home that measures their oxygen level (SpO_2) and breathing during the night. In our project we simulate five patients.

System: The device data goes through Kafka (3 brokers, so one can fail) to an AI service that finds apnea events. Then we calculate the AHI (apnea-hypopnea index, events per hour) and sort the patient into normal, mild, moderate or severe. The doctor sees the results on a dashboard and approves each one. If the oxygen level drops too low, the doctor gets an alert right away.

Why we chose this topic: It is a real medical problem and easy to explain. The night analysis is not urgent, it only has to be ready by 08:00 in the morning. So the system can wait for electricity with lower CO_2 and choose between a fast and an accurate model. Alerts are the exception and always run immediately.

Out of scope: login for patients and doctors, history over several nights, sending reports to patients, therapy monitoring.

Use cases

#	Use case	Actor	Description
UC1	Send night data	Device (simulator)	The simulator sends the SpO_2 and breathing data of a night minute by minute over Kafka. The raw night is saved in the database.
UC2	Analyse a night	System	The system finds apnea events with the fast or the accurate model (depending on the carbon mode), calculates the AHI and severity and saves the result. It also measures how much CO_2 the analysis used.
UC3	Raise an alert	System, doctor	If SpO_2 goes below 80 %, an alert shows up on the doctor dashboard right away. Alerts never wait.
UC4	Review a result	Doctor	The doctor sees the patients sorted by severity, opens a report with the SpO_2 chart and approves or corrects the result. Every decision is logged.
UC5	Adapt to carbon intensity	Carbon scheduler	With high carbon intensity (simulated), the analysis waits for cleaner electricity but is done before 08:00; close to the deadline it uses the fast model. It can also move to a node in a cleaner (simulated) region. With low carbon intensity, the accurate model is used.
UC6	Scale under load	Kubernetes	When a lot of nights come in at the same time, Kubernetes starts more apnea-detector instances and removes them again later.
UC7	Monitor the system	Team (developers)	A dashboard shows response times, running instances, which model was used and how much CO_2 each analysis needed.

Microservices

Microservice	Use cases	What it does	Its data
device-simulator	UC1, UC6	Creates one night of SpO_2 and breathing data for each of the 5 patients and sends it to Kafka.	– (synthetic data)
apnea-detector	UC1, UC2, UC3, UC5, UC6	Cleans the data, saves the raw night, finds apnea events with the fast or accurate model, measures CO_2 and sends alerts.	PostgreSQL: raw_nights
ahi-triage	UC2	Calculates the AHI and severity for each night and saves the result.	PostgreSQL: results
alert-service	UC3	Gets alerts over ZeroMQ and saves them for the dashboard.	PostgreSQL: alerts
doctor-dashboard	UC3, UC4	Shows patients, SpO_2 charts and alerts; the doctor approves or corrects results.	PostgreSQL: decisions (audit log)
carbon-scheduler	UC5	Reads the (simulated) carbon intensity, sets low or high carbon mode, delays the analysis and moves the detector.	etcd: carbon mode

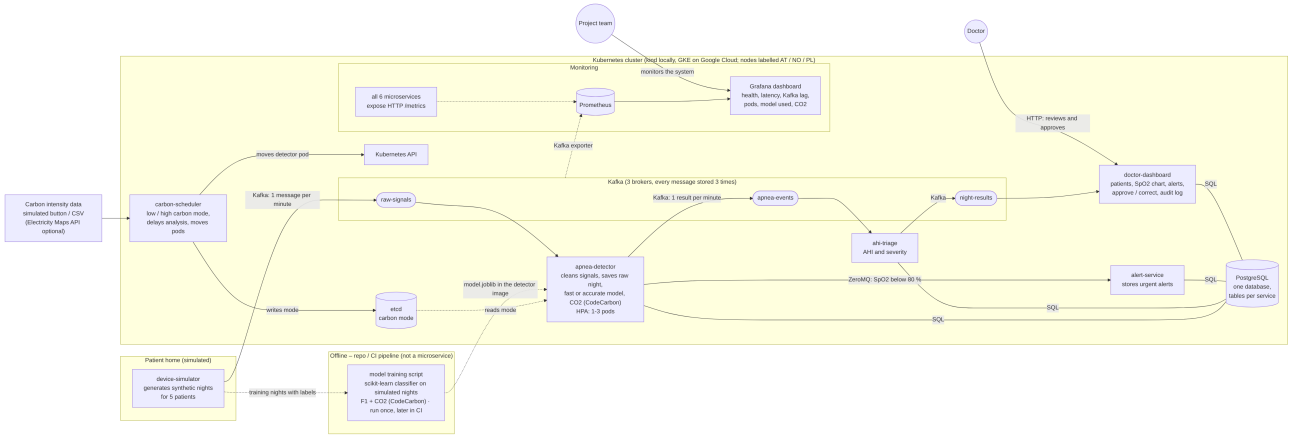
AI model: The AI model is not a separate microservice: it is part of the apnea-detector, which loads a scikit-learn model that we train beforehand with a training script on simulated nights.

Questions for the professor

1. Is it OK to have one PostgreSQL database with five tables instead of one database per microservice? We simplified this for the demo: each table is written by only one service, so the services are still separated.

Diagrams

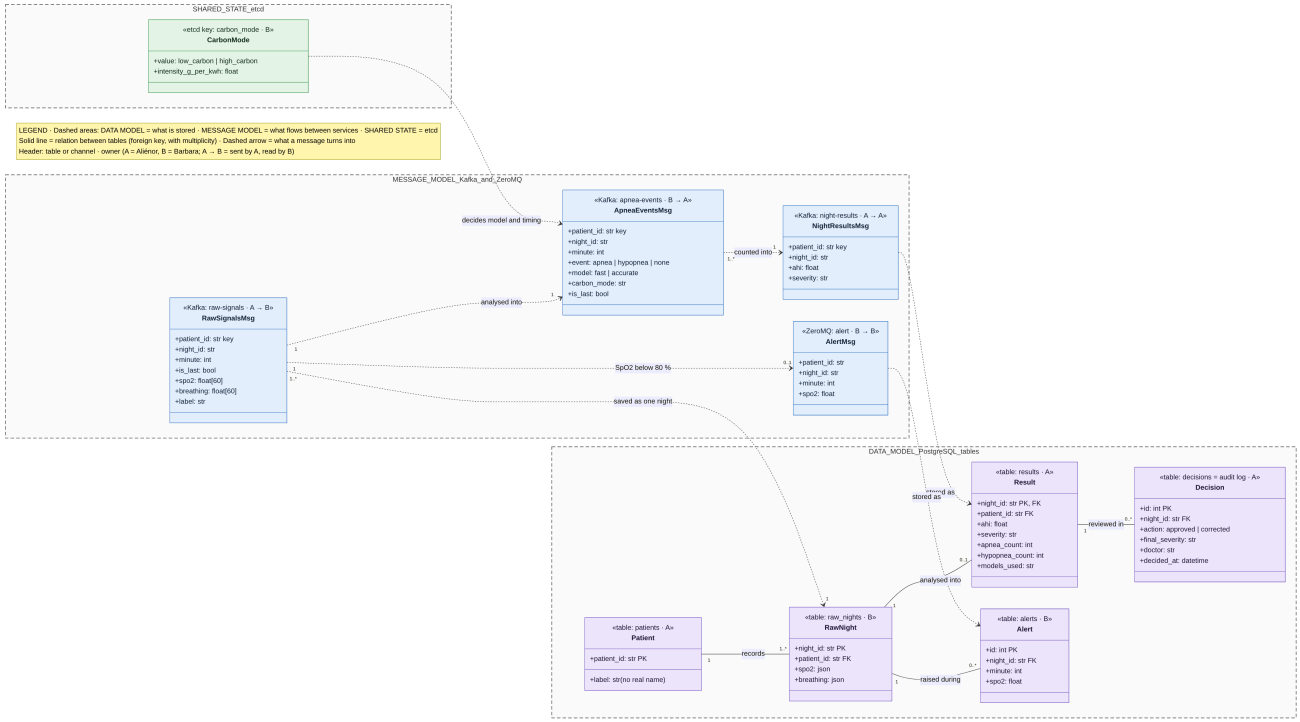
Component diagram



We have six microservices: the device-simulator stands for the patients' homes and the other five run in Kubernetes. They mostly talk through three Kafka topics (raw-signals, apnea-events, night-results) on a Kafka cluster with 3 brokers, where every message is stored three times and waits until it is processed; alerts go over ZeroMQ, the carbon mode is kept in etcd, the stored data is in 1 PostgreSQL database with 5 tables, and Prometheus and Grafana collect the metrics.

Full size in our repo: diagrams/30.10_B_component-diagram_v3.md

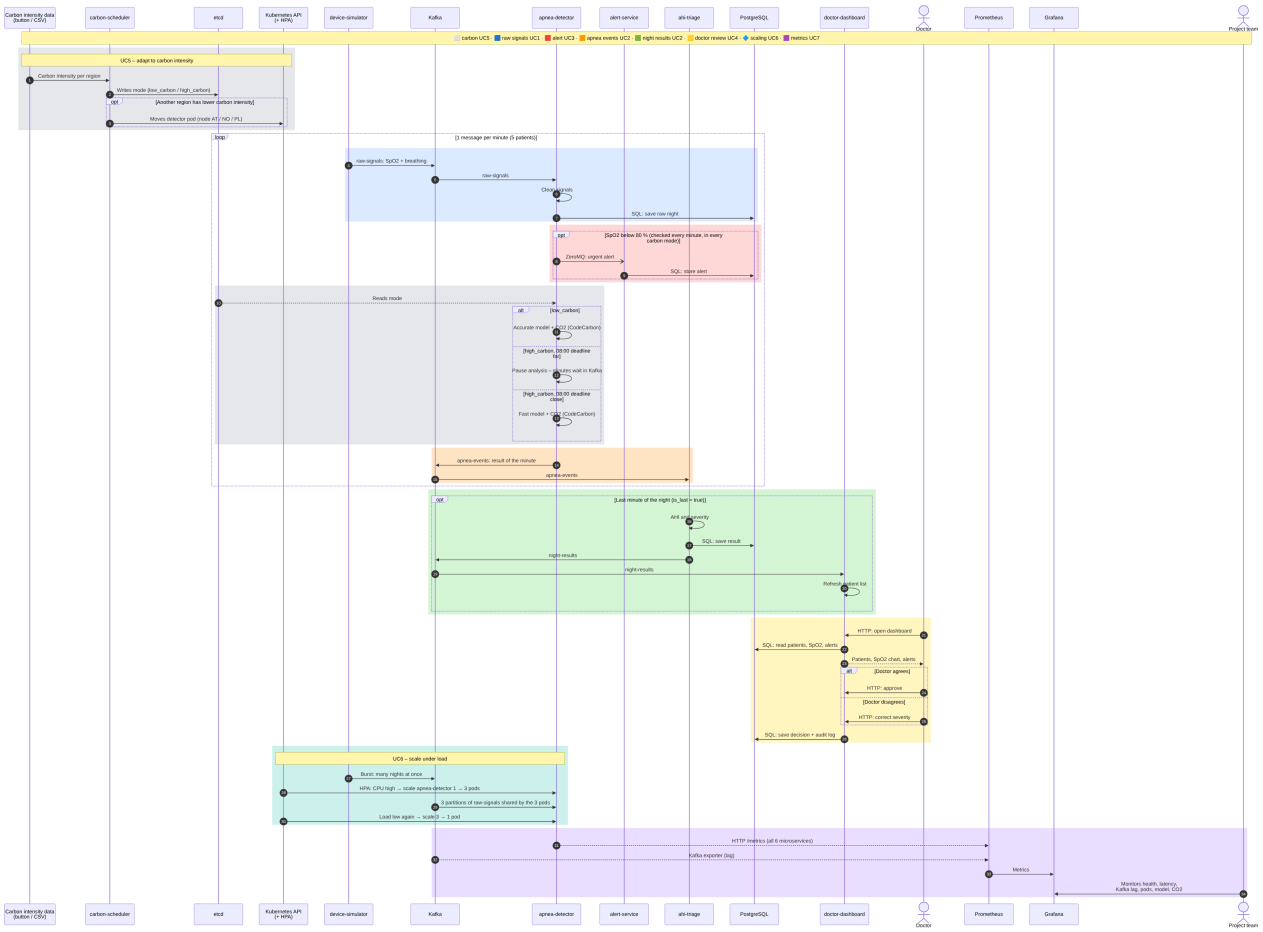
Data model and message model



We only need five simple tables in one PostgreSQL database (patients, raw_nights, results, alerts, decisions), and each table is written by just one service. Between the services we send four kinds of messages: three over Kafka (raw-signals, apnea-events, night-results) and the alert over ZeroMQ.

Full size in our repo: diagrams/30.10_B_data-message-model_v1.md

Sequence diagram – one night end to end (UC1–UC7)



The night comes in minute by minute: the apnea-detector checks every minute for a dangerous SpO_2 drop, and the carbon mode decides if it runs the accurate model, pauses the analysis or uses the fast model close to 08:00. After the last minute, ahi-triage calculates the AHI and the doctor approves or corrects it, and when many nights come at once Kubernetes scales the detector from 1 to 3 pods.

Full size in our repo: diagrams/30.10_B_sequence-diagram_v2.md